

摘要：

阿里云在云栖大会发布企业级Agent平台AgentCore，针对个人AI工具提升效率却未能同步提升企业协作效率的痛点，提供统一构建、纳管与治理能力。平台应对三大挑战：工程化方面任务完成率超99%；规模化方面覆盖主流Agent框架，TCO可降70%；协作化方面通过“岗位角色”实现人机原生协作。

个人生产力提升了，等于企业的生产力也提升了吗？似乎，大家比以前更忙了。

我们用 AI Coding 写代码，用办公 Agent 处理各类文档需求。个人生产力被大幅提升，每个岗位都在左移和右移，越来越多的人成为企业内的多面手。**但是员工间的协作效率并没有提升，反而因为每个人生产力提升带来的协作中间产物增多，导致协作负担变重。**你用 AI 加大输出，我再用 AI 简化输入。

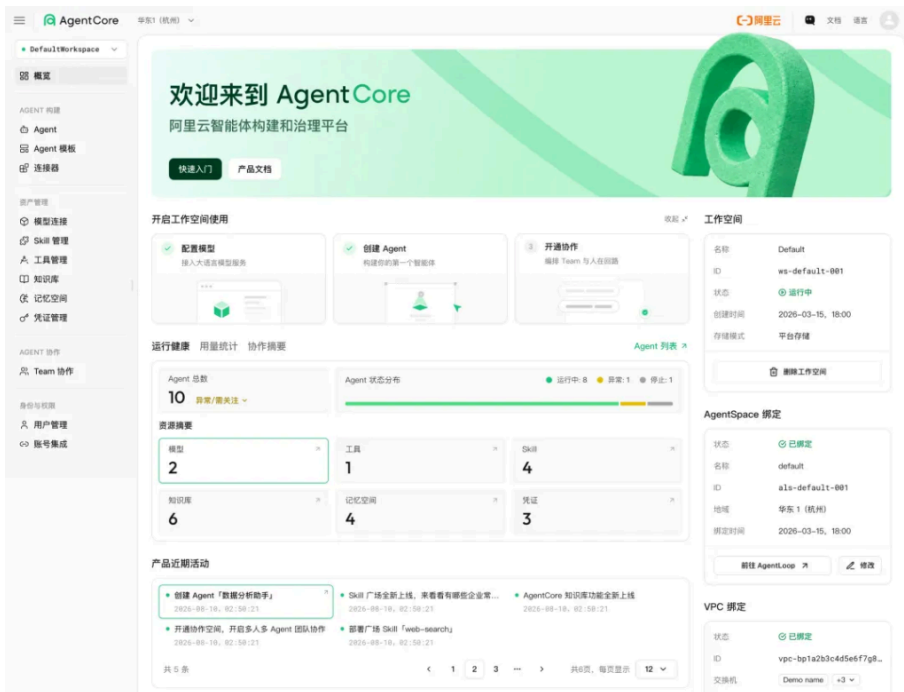
软件的本质，是解决规模化带来的复杂度问题。企业不止是个人的简单叠加，还涉及目标、理解、协作、组织设计等等，企业生产力的提升，复杂度远高于个人生产力的提升。

云栖大会期间，阿里云发布了企业级 Agent 平台 AgentCore，旨在通过更加原生的人机协作方式，以及更完整的 Agent 构建和治理的平台能力，提升企业的生产力。

阿里云 AgentCore 是什么？

阿里云 AgentCore 服务的对象是企业，由企业 AI 中台或类似定位的技术团队基于 AgentCore 构建新的智能体、统一治理企业内的多源智能体，包括员工使用的 Coding 和办公智能体、各部门团队自建的智能体、外采的智能体服务等。

从下方控制台界面看，是作为平台型产品，AgentCore 围绕智能体生命周期提供了构建、运行、治理和调优的能力。



此外，不同于其他 Agent 平台型产品，AgentCore 还提供了 Team 协作空间，使用 IM 这种更加原生的协作方式，通过创建“岗位角色”，组织长期工作，而非通过“任务”来满足短期需求。



智能体成为企业生产力，有哪些挑战？

Coding Agent 和办公 Agent 加速了企业对智能体的认知，但是智能体要成为企业生产力，依旧存在以下三大挑战：

- 工程化挑战：从概率智能到可靠生产力，Agent 需要能够承担关键任务。
- 规模化挑战：稳定、安全、性能、成本，从单点试验到智能基础设施，Agent 需要能够被大规模部署。
- 组织化挑战：从 Agent 孤岛到智能组织，Agent 需要能够进入核心业务流程。



企业生产力的进化，经历了三个阶段：个人经验、流程和软件、智能化能力。我们目前正处于智能化能力的前期。当 Agent 真正进入生产链路、承担真实工作，需要重新定义组织的流程、角色、协作与度量方式。

例如，在构建与准入阶段，企业要对多源智能体进行身份和鉴权、隔离环境执行、网关和外部系统管控。在组织与交付阶段，人机协作中要设计任务拆解、协作规则、分配上下文、成员角色管理和结果汇总，并要求过程可干预。评测与进化阶段，要做智能体调优，例如数据接入、智能体轨迹、评估目标和任务设定、实验回测等。沉淀与复用阶段，要构建企业智能资产，包括 Skills 与 workflow、知识与记忆、工具契约，评测集与质量基线等。

AgentCore 如何应对这些挑战？



AgentCore 以统一构建、统一纳管、统一治理三层能力，加上企业智能资产广场，覆盖上述生命周期。对智能体运行所需的基础组件进行统一构建，把不同技术栈的智能体接入同一套运行与管控体系进行统一纳管，对身份、权限、观测、评测与审计提供一致策略，负责企业自有 Skill、MCP 与 Agent 的登记与分发。



应对工程化挑战：让 Agent 承担关键任务

Agent = Model + Harness，大量事实证明，Agent 承担关键任务不可靠，并不是因为模型智力问题，而是不恰当的工程设计导致的。

企业关键任务通常运行时间长、涉及多次工具调用和执行分支，AgentCore 的 Harness 层通过管理上下文、模型、工具、持久状态与故障恢复，以及运行时权限，使任务在中断后可以恢复而不是从头重来；所有执行发生在运行时沙箱中，代码沙箱、浏览器沙箱与通用沙箱提供会话隔离与弹性运行。**任务完成率超 99%。**

在此之上，AgentCore 把每次执行变成可验证的证据链。Trace 记录任务轨迹与工具调用，配合用户反馈定位失败发生在哪一步，**线上故障响应时间缩短至 1s 以内**。评估环节由具备工具与领域知识的 Agent Judge 进行自动化评估，经测试，**准确性 >90% 人类专家**，同时保留必要的人工复核。

失败案例、上下文与专家结论，可以在平台上继续沉淀为版本化 Dataset，对 Prompt、Skill、模型与上下文的调整都经过实验、回归和灰度验证后再发布。企业由此可以判断一个 Agent 是否达到承担关键任务的标准，而不是依据演示效果做决定。

应对规模化挑战：让 Agent 被大规模部署

规模化的前提是不为每个团队、每种技术栈重复建设。AgentCore 统一构建把智能体运行所需的基础组件收敛为共享底座：运行时沙箱提供隔离执行环境，记忆库提供长短期记忆并可对接多个记忆服务，知识库以 RAG 方式接入百炼与 RAGFlow，模型服务支持多模型统一接入并兼容百炼与 BYOK，网关统一接入并管控 OpenAPI 与 MCP 调用。

统一纳管让高代码 Agent（LangChain、LangGraph、Google ADK、AgentScope）、开源 Harness Agent（OpenClaw、QwenPaw）与商业化 SaaS Agent（Claude Code、Codex、Qoder、QoderWork）进入同一套运行与治理体系，自研、托管与外部引入的智能体不必各建一套配套能力，**100% 覆盖主流 Agent。**

稳定、安全、性能与成本在同一套治理策略下约束。AgentCore 拥有独立、可管理的工作负载身份，按最小权限与联合授权承接任务并访问能力；调用外部系统经由网关完成凭证转换、访



问控制与调用额度控制，**越权拦截率超 99%**；运行时权限与预算把成本约束前置到执行环节，例如临时探索的分支可以在完成后释放，稳定任务可以形成按需编队，长期稳定的能力可以注册为可发现、可调用的服务，生命周期由任务特点决定，可以显著降低 Agent 长时任务中的闲置算力成本，**TCO 下降可达 70%**，观测与审计保留完整证据，可以追溯谁发起了任务、使用什么身份、调用了哪个工具、影响了哪些资源，**支持面向长任务秒级审计**，并为容量规划与成本归属提供依据。模型服务的多模型接入使企业能够按任务难度与成本要求选择模型，而不是让所有任务使用同一档能力。

应对协作化挑战：让 Agent 进入核心业务流程

Agent 进入核心业务流程，要同时解决协作与责任。AgentCore 提供原生的协作方式，通过创建“岗位角色”，组织长期工作，而非通过“任务”来满足短期需求。以构建一个 AI 用量与成本分析平台的 Web 应用为例，在 AgentCore Team 协作中设定产品经理、研发、测试、运维的 Agent 岗位角色，再通过 Leader Agent 拆解任务、分配上下文并汇总证据，日志调查、链路调查与变更校验 Agent 并行推进；任务之间隔离上下文，同一任务内共享必要信息与结论；人定义目标、补充信息、确认关键操作，并在异常时接管。我们来看一段多人多 Agent 的协作视频。

应对工程化挑战：让 Agent 承担关键任务

Agent = Model + Harness，大量事实证明，Agent 承担关键任务不可靠，并不是因为模型智力问题，而是不恰当的工程设计导致的。

企业关键任务通常运行时间长、涉及多次工具调用和执行分支，AgentCore 的 Harness 层通过管理上下文、模型、工具、持久状态与故障恢复，以及运行时权限，使任务在中断后可以恢复而不是从头重来；所有执行发生在运行时沙箱中，代码沙箱、浏览器沙箱与通用沙箱提供会话隔离与弹性运行。任务完成率超 99%。

在此之上，AgentCore 把每次执行变成可验证的证据链。Trace 记录任务轨迹与工具调用，配合用户反馈定位失败发生在哪一步，线上故障响应时间缩短至 1s 以内。评估环节由具备工具与领域知识的 Agent Judge 进行自动化评估，经测试，准确性 >90% 人类专家，同时保留必要的人工复核。失败案例、上下文与专家结论，可以在平台上继续沉淀为版本化 Dataset，对 Prompt、Skill、模型与上下文的调整都经过实验、回归和灰度验证后再发布。企业由此可以判断一个 Agent 是否达到承担关键任务的标准，而不是依据演示效果做决定。

应对规模化挑战：让 Agent 被大规模部署

规模化的前提是不为每个团队、每种技术栈重复建设。AgentCore 统一构建把智能体运行所需的基础组件收敛为共享底座：运行时沙箱提供隔离执行环境，记忆库提供长短期记忆并可对接多个记忆服务，知识库以 RAG 方式接入百炼与 RAGFlow，模型服务支持多模型统一接入并兼容百炼与 BYOK，网关统一接入并管控 OpenAPI 与 MCP 调用。统一纳管让高代码 Agent（LangChain、LangGraph、Google ADK、AgentScope）、开源 Harness Agent（OpenClaw、QwenPaw）与商业化 SaaS Agent（Claude Code、Codex、Qoder、QoderWork）进入同一套运行与治理体系，自研、托管与外部引入的智能体不必各建一套配套能力，100% 覆盖主流 Agent。

稳定、安全、性能与成本在同一套治理策略下约束。AgentCore 拥有独立、可管理的工作负载身份，按最小权限与联合授权承接任务并访问能力；调用外部系统经由网关完成凭证转换、访问控制与调用额度控制，越权拦截率超 99%；运行时权限与预算把成本约束前置到执行环节，例如临时探索的分支可以在完成后释放，稳定任务可以形成按需编队，长期稳定的能力可以注册为可发现、可调用的服务，生命周期由任务特点决定，可以显著降低 Agent 长时任务中的闲

置算力成本，TCO 下降可达 70%，观测与审计保留完整证据，可以追溯谁发起了任务、使用什么身份、调用了哪个工具、影响了哪些资源，支持面向长任务秒级审计，并为容量规划与成本归属提供依据。模型服务的多模型接入使企业能够按任务难度与成本要求选择模型，而不是让所有任务使用同一档能力。

应对协作化挑战：让 Agent 进入核心业务流程

Agent 进入核心业务流程，要同时解决协作与责任。AgentCore 提供原生的协作方式，通过创建“岗位角色”，组织长期工作，而非通过“任务”来满足短期需求。以构建一个 AI 用量与成本分析平台的 Web 应用为例，在 AgentCore Team 协作中设定产品经理、研发、测试、运维的 Agent 岗位角色，再通过 Leader Agent 拆解任务、分配上下文并汇总证据，日志调查、链路调查与变更核验 Agent 并行推进；任务之间隔离上下文，同一任务内共享必要信息与结论；人定义目标、补充信息、确认关键操作，并在异常时接管。我们来看一段多人多 Agent 的协作视频。

平台能力和企业 AI 资产的乘数效应

如果说企业生产力是产出，那么和平台如何分工就是企业的投入策略了。

过去一两年，Agent 开发的变化速度，可能超过了很多团队的预期。不少团队都有类似经历：一开始自己写 Agent Loop，后来接入 LangChain、LangGraph 等框架，再后来又开始关注模型公司提供的 Agent SDK 和 Harness Runtime。每一次模型能力升级，似乎都伴随一次架构调整。

以前需要业务团队自己实现的 Planner、Memory、Router、重试和状态管理，正在逐渐变成模型或平台的通用能力。于是，一个现实问题摆在面前：如果模型、Harness 还会继续变化，今天投入建设的 Agent，哪些部分能够真正留下来？

Agent 的建设可以拆成两条增长轴：



横轴代表通用智能，包括 Memory、Planner、Managed Runtime、Agent 协作等能力。这些能力是模型公司和云平台持续投入的方向，企业应该跟进，并尽量复用，避免重复制造已经逐渐成为基础设施的东西。横轴上的能力，未必适合作为企业长期资产。

纵轴代表业务深度，包括企业持续产生的业务事实，业务对象之间的关系，企业内部统一的业务语义，具体流程和规则，行业知识与业务经验。这些内容不会因为模型和云平台升级替企业完成，仍然需要企业在业务侧沉淀。

模型公司沿着通用智能的方向不断向前，企业沿着业务深度的方向持续向上，获得的智能面积越大，企业生产力的 ROI 越高。

Skills 与 workflow、知识与记忆、工具契约、评测集与质量基线，这些都是企业的 AI 资产，企业基于平台持续的构建、优化和沉淀，模型越强，Agent 基于这些资产的推理和行动，就会更可靠。



风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

117 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论